AWS Multi-Tier Frontend & Load Balancer Architecture Documentation

Project Title

Highly Available Frontend, Load Balancing, Security, and Auto Scaling Architecture on AWS.

Objective

Deploy a highly available and scalable frontend application using:

- **S3 Static Website Hosting**
- **Application Load Balancer (ALB)**
- **Network Load Balancer (NLB)** (Theory)
- **Gateway Load Balancer (GWLB)** (Theory)
- **EC2 instances in Spread Placement Group**
- **Lambda Integration**
- **Auto Scaling with Step Scaling Policies**

AWS Services Used

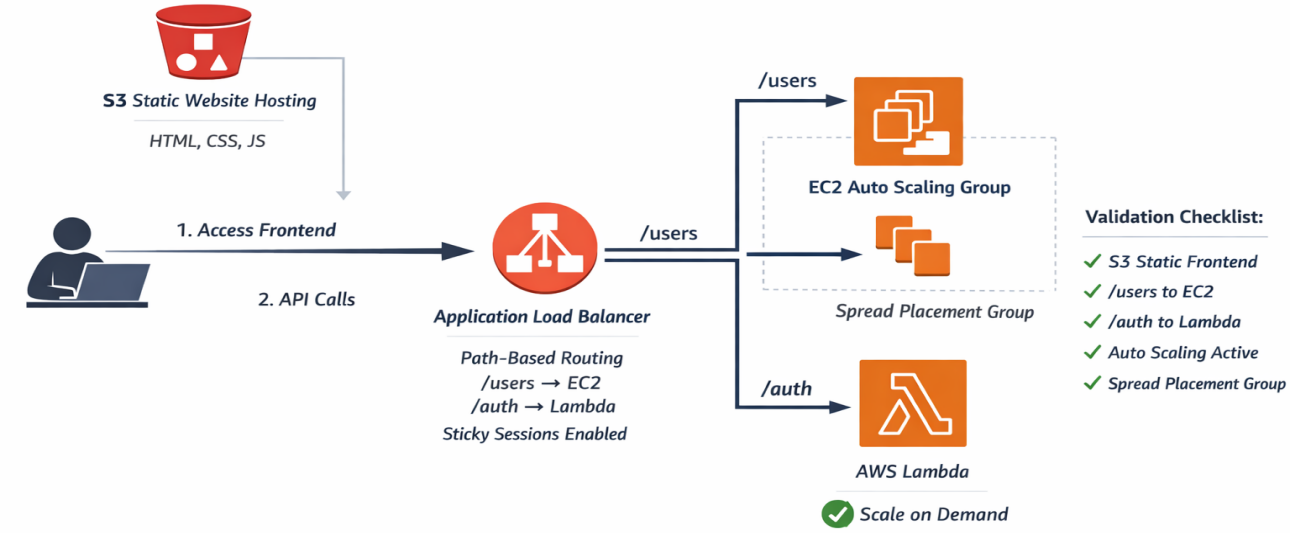
Service	Purpose
Amazon S3	Static website frontend hosting
ALB	HTTP path-based routing
NLB	Static public IP exposure
GWLB	Traffic inspection through firewall
EC2	Backend API fleet
Lambda	Authentication service
Auto Scaling	Dynamic scaling
CloudWatch	Metrics & alarms
ENI	Monitoring interface

AWS Services Covered

- Amazon S3
- EC2
- Application Load Balancer (ALB)
- AWS Lambda
- Auto Scaling Group
- CloudWatch
- Placement Group
- Secondary ENI
- Network Load Balancer (Theory)
- Gateway Load Balancer (Theory)

Architecture Overview

```
User
   ↓
S3 Static Frontend
   ↓ API Calls
Application Load Balancer
   ├── /users → EC2 Backend
   └── /auth → Lambda
   ↓
Auto Scaling Group
```



Key Concepts Tested

- S3 Static Website Hosting
- CORS
- ALB Path-Based Routing
- Sticky Sessions
- Cross-Zone Load Balancing
- Lambda Integration
- Spread Placement Group
- Secondary ENI
- Step Scaling
- NLB Static IP (Theory)
- GWLB Firewall Routing (Theory)

Validation Criteria

✓ S3 frontend loads successfully
✓ API requests reach ALB
✓ /users routes to EC2
✓ /auth routes to Lambda
✓ Auto scaling works
✓ Spread placement configured
✓ Secondary ENI attached
✓ NLB and GWLB concepts documented

STEP 1 — CREATE S3 FRONTEND BUCKET

Go to:

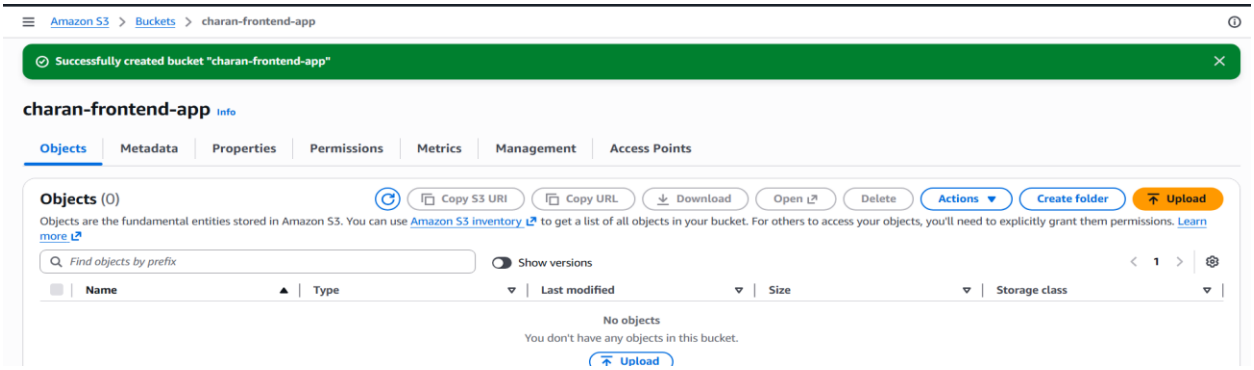
AWS Console → S3 → Create Bucket

Use bucket name:

charan-frontend-app

Important Settings

- Uncheck **Block all public access**
- Keep default region



Enable Static Website Hosting

Open:

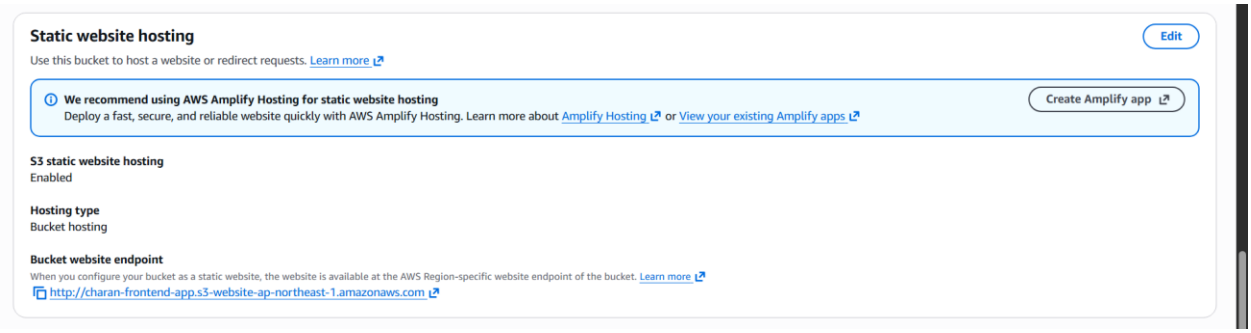
S3 → Bucket → Properties

Enable:

Static website hosting

Set:

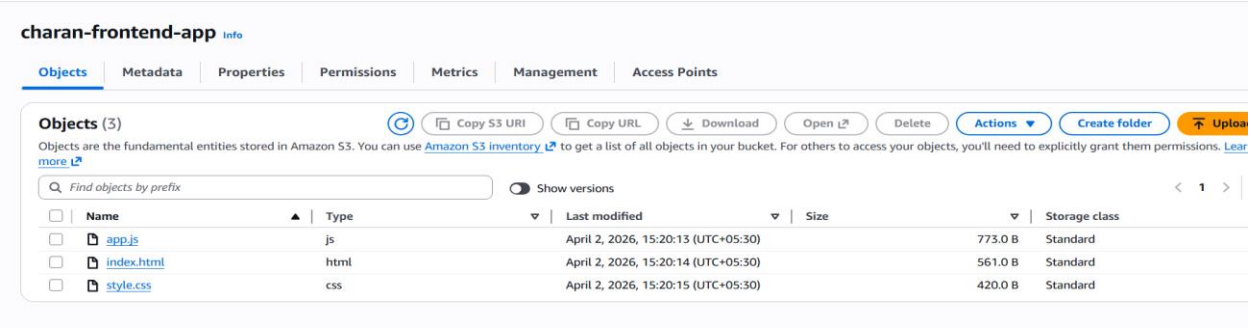
Index document: index.html
Error document: error.html



Upload Frontend Files

Upload:

index.html
style.css
app.js



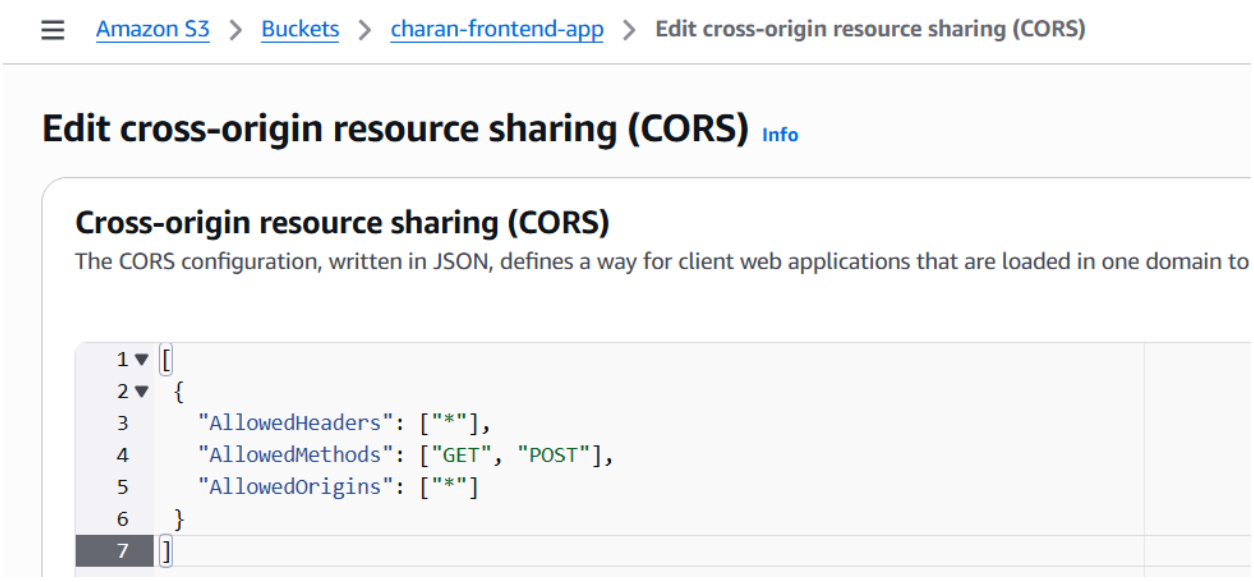
Configure CORS

Go to:

Permissions → CORS

Paste:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "POST"],
    "AllowedOrigins": ["*"]
  }
]
```



STEP 2 — CREATE LAMBDA FUNCTION FOR AUTH

Go to:

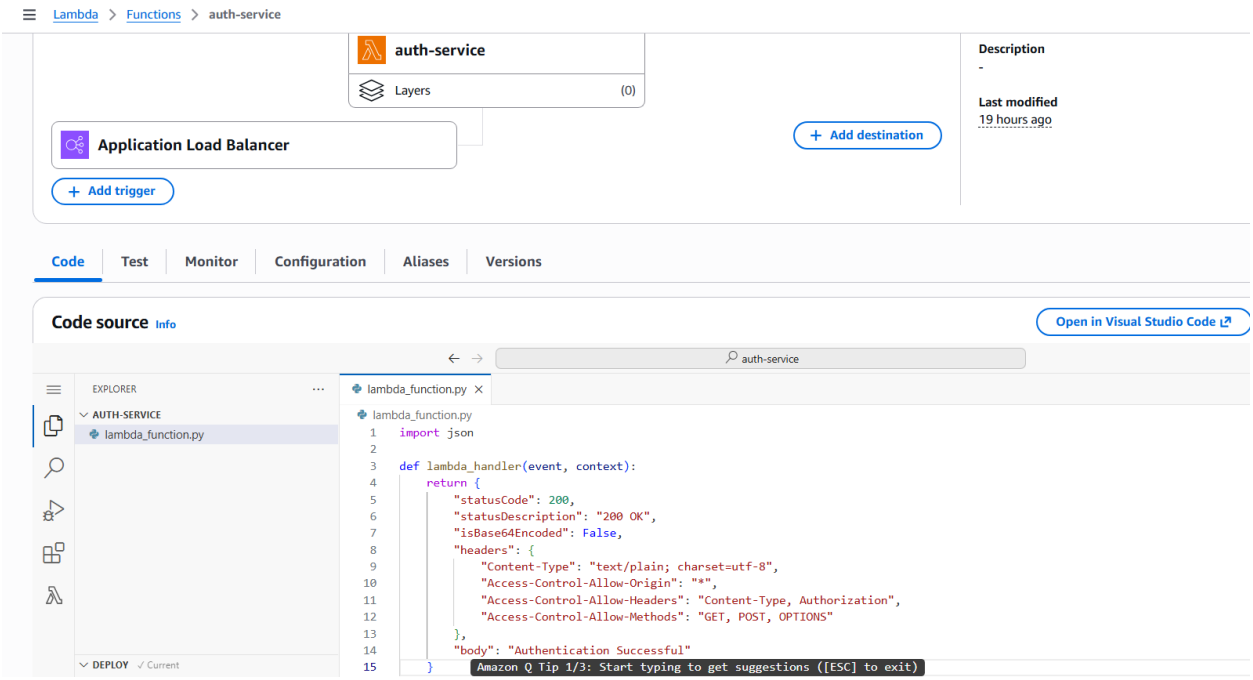
Lambda → Create Function
Settings:

Name: auth-service
Runtime: Python 3.12
Paste code:

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Headers": "Content-Type",
            "Access-Control-Allow-Methods": "GET, POST, OPTIONS"
        },
```

```
        "body": "Authentication Successful"
    }
```

Click **Deploy**



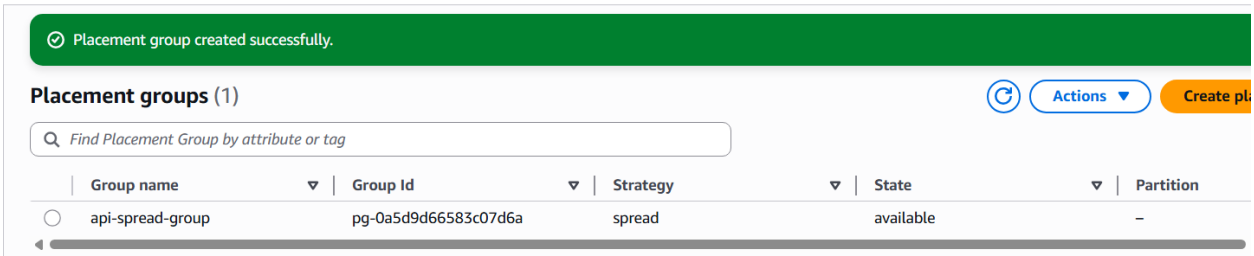
STEP 3 — CREATE SPREAD PLACEMENT GROUP

Go to:

EC2 → Placement Groups → Create
Select:

Spread
Name:

api-spread-group
This ensures instances are placed on **separate hardware racks.**



STEP 4 — LAUNCH EC2 BACKEND INSTANCE

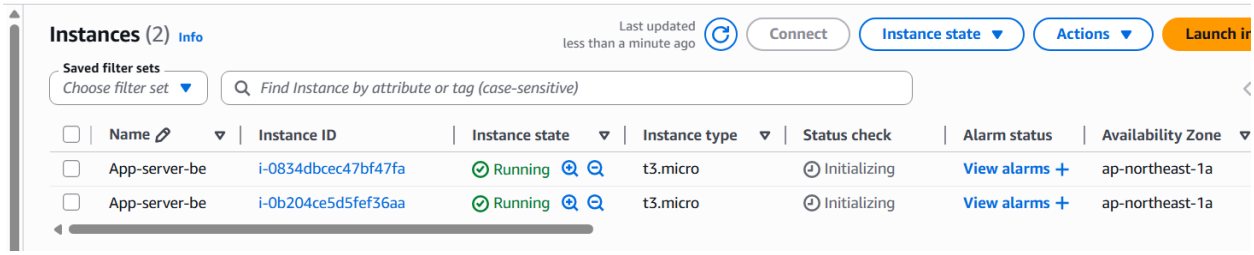
Go to:

EC2 → Launch 2 Instances
Use:

Amazon Linux 2
t2.micro / t3.micro

Placement Group:

api-spread-group



Install Backend API

SSH into instance:

```
sudo yum update -y
sudo yum install python3 –y

sudo yum install python3-pip -y
```

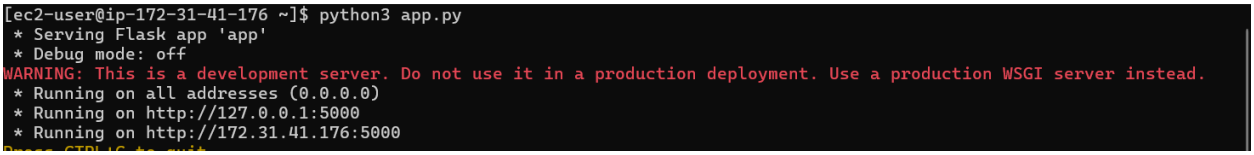
```
pip3 install flask
```

Create app.py

```
from flask import Flask, jsonify

app = Flask(name)

@app.after_request def add_cors_headers(response):
response.headers["Access-Control-Allow-Origin"] = "*"
response.headers["Access-Control-Allow-Headers"] = "Content-Type"
response.headers["Access-Control-Allow-Methods"] = "GET, POST,
OPTIONS" return response

@app.route('/') def home(): return "Backend API Running"

@app.route('/users') def users(): return jsonify({ "status":
"success", "users": [ {"id": 1, "name": "Charan"}, {"id": 2, "name":
"DevOps User"} ] })

if name == 'main': app.run(host='0.0.0.0', port=5000)
```

Run:

```
python3 app.py
```



STEP 5 — ATTACH SECONDARY ENI

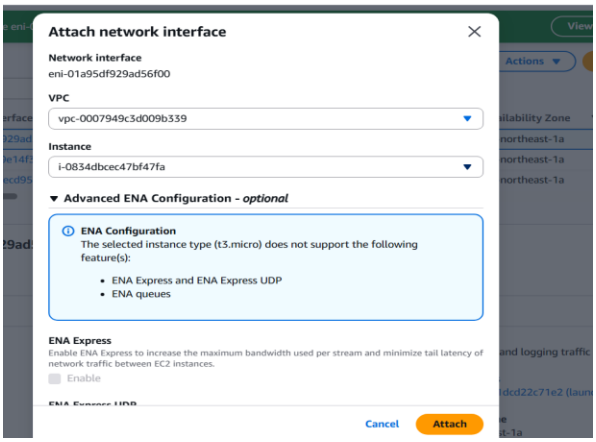
Go to:

EC2 → Network Interfaces → Create

Attach to the instance.

Purpose:

Monitoring and logging traffic



STEP 6 — CREATE APPLICATION LOAD BALANCER

Go to:

EC2 → Load Balancers → Create → Application Load Balancer

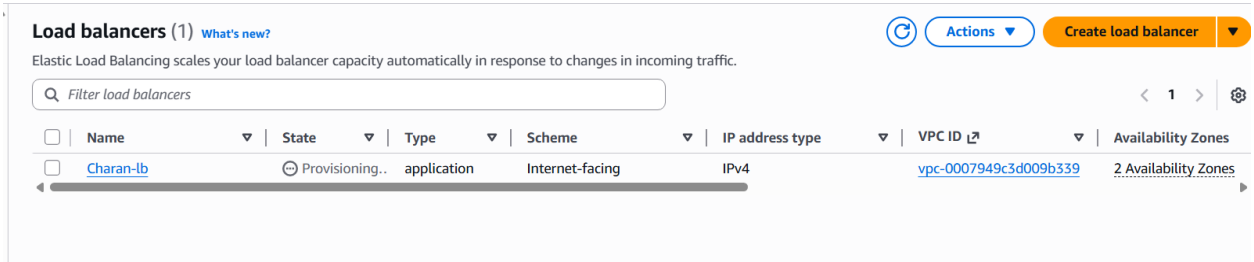
Select:

Internet-facing

Enable:

Sticky sessions

Cross-zone load balancing

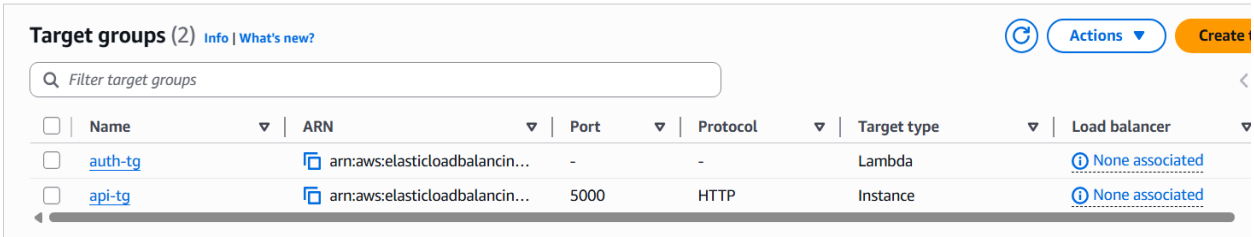


Create Target Groups

EC2 Target Group

Target Type: Instances
Register EC2 instance.

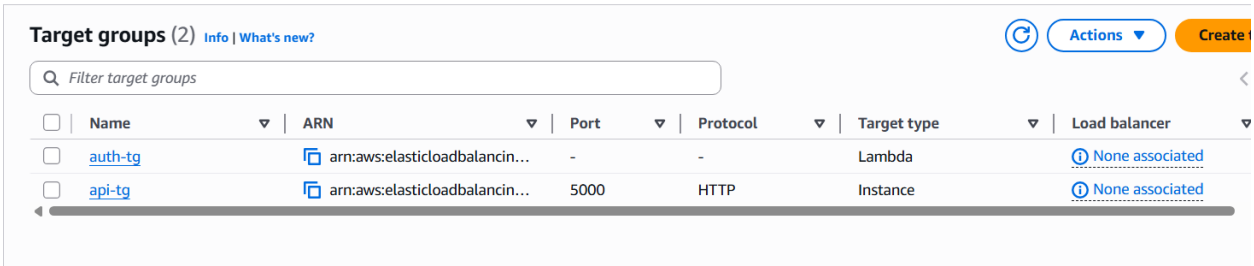
Name:api-tg



Lambda Target Group

Target Type: Lambda
Select:

Name:auth-tg



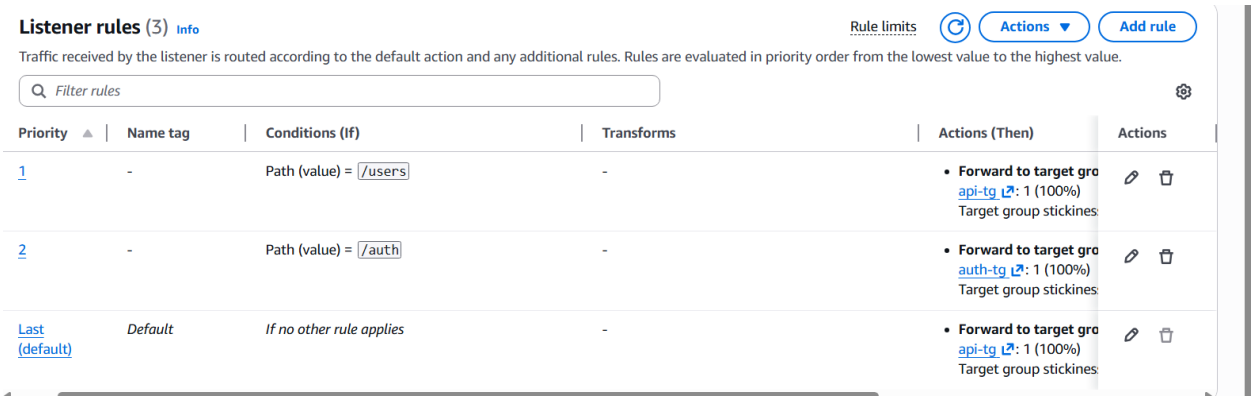
STEP 7 — CONFIGURE PATH-BASED ROUTING

Configure ALB listener rules:

/users → EC2 Target Group
/auth → Lambda Target Group

This ensures:

- /users → backend API
- /auth → Lambda auth service



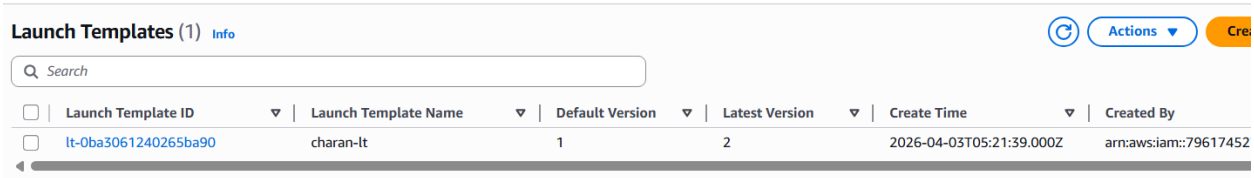
STEP 8 — CONFIGURE AUTO SCALING

Create Launch Template

Go to:

EC2 → Launch Templates

Use current EC2 instance configuration.

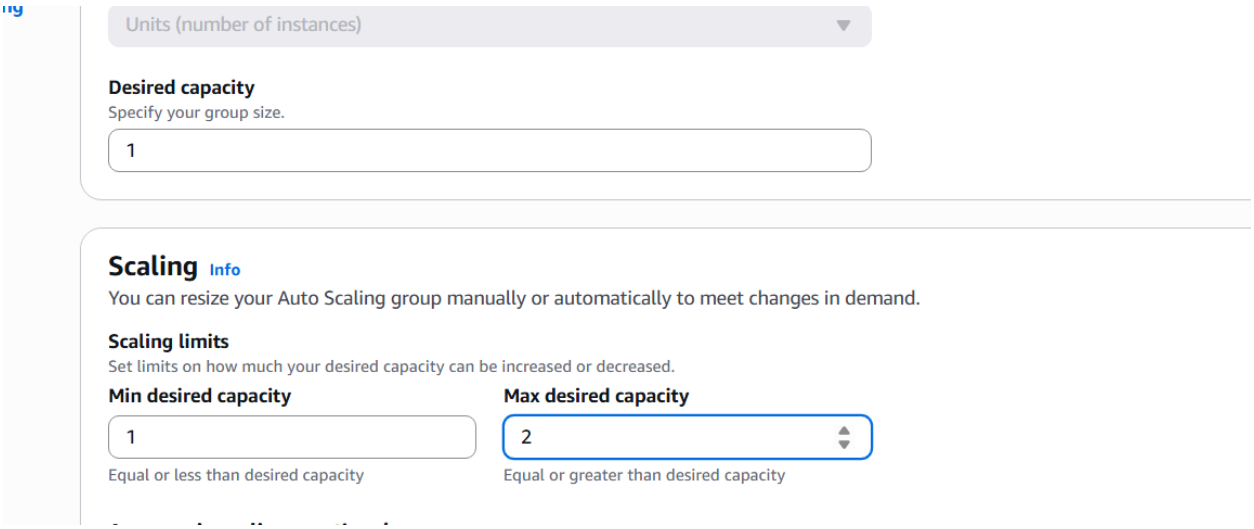


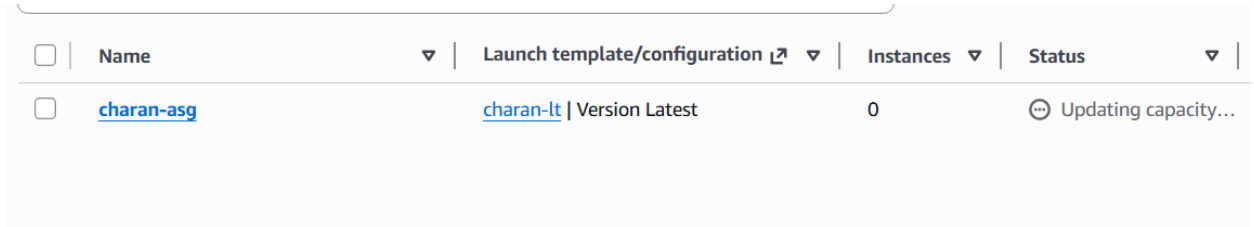
Create Auto Scaling Group

Use:

Minimum: 1
Desired: 1
Maximum: 2
Attach:

ALB target group
Placement group





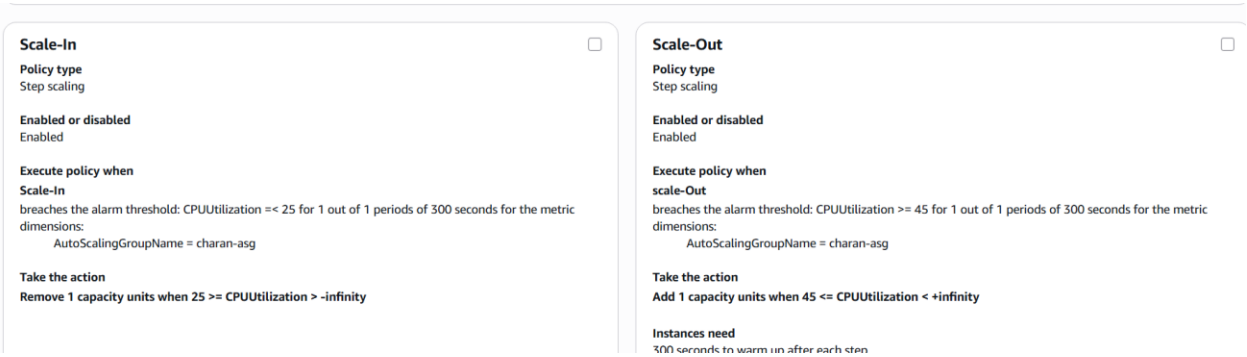
Configure Step Scaling

Policy 1:

+1 instance if CPU > 50%

Policy 2:

-1 instance if CPU < 25%



Scale-In

Policy type
Step scaling

Enabled or disabled
Enabled

Execute policy when
Scale-In
breaches the alarm threshold: CPUUtilization =< 25 for 1 out of 1 periods of 300 seconds for the metric dimensions:
AutoScalingGroupName = charan-asg

Take the action
Remove 1 capacity units when 25 >= CPUUtilization > -infinity

Scale-Out

Policy type
Step scaling

Enabled or disabled
Enabled

Execute policy when
scale-Out
breaches the alarm threshold: CPUUtilization >= 45 for 1 out of 1 periods of 300 seconds for the metric dimensions:
AutoScalingGroupName = charan-asg

Take the action
Add 1 capacity units when 45 <= CPUUtilization < +infinity

Instances need
300 seconds to warm up after each step

STEP 9 — OPTIONAL THEORY: NETWORK LOAD BALANCER (NLB)

Theory only — not deployed in Free Tier implementation

NLB works at:

Layer 4 → TCP / UDP / TLS

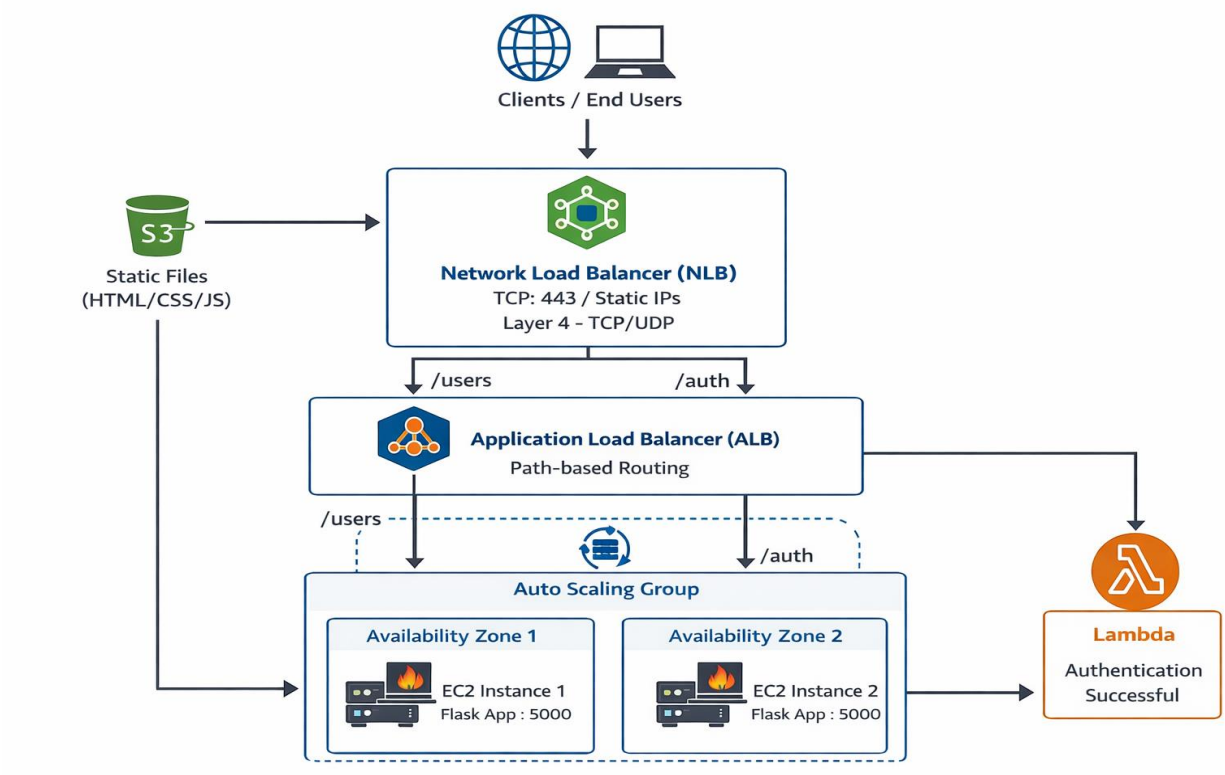
Used for:

- static IP addresses
- low latency traffic
- high performance APIs

Example use:

User → NLB → ALB → EC2
Use case:

TCP:443
Elastic IP per AZ



STEP 10 — OPTIONAL THEORY: GATEWAY LOAD BALANCER (GWLB)

Theory only — not deployed in Free Tier

GWLB is used for:

firewall appliances
packet inspection
IDS / IPS
traffic filtering

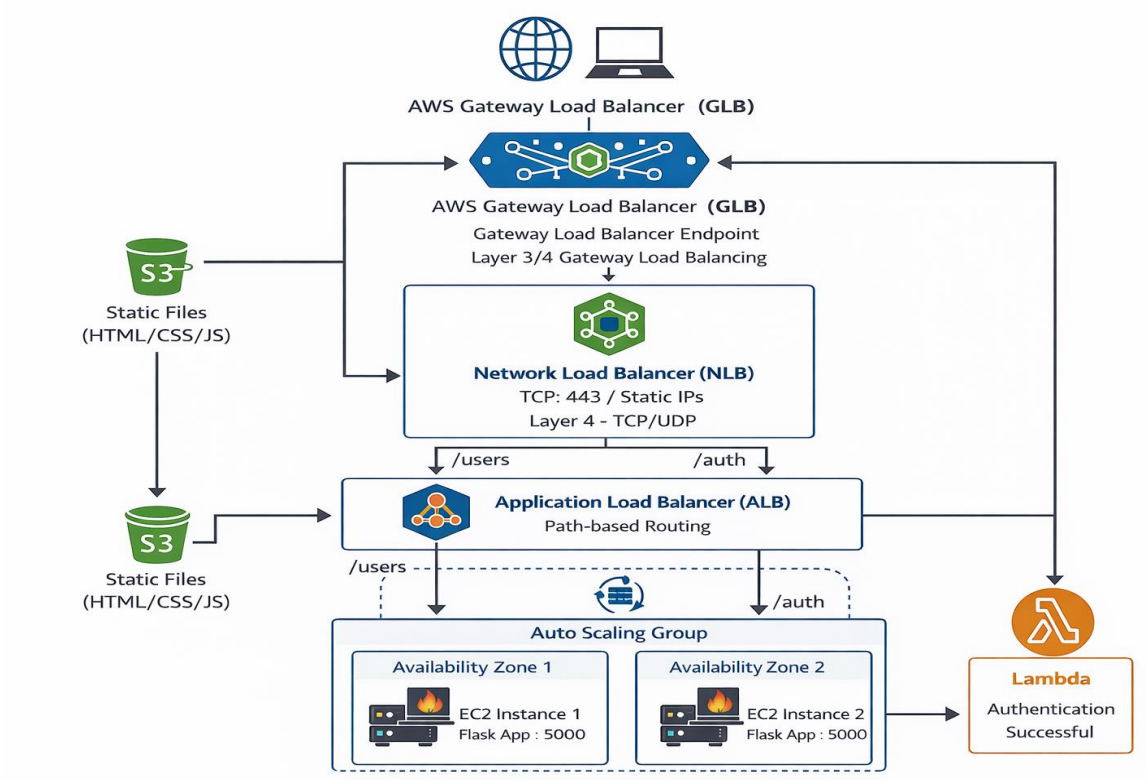
Protocol:

GENEVE : 6081

Traffic flow:

User → GWLB → Firewall → ALB → EC2

Used for enterprise security.

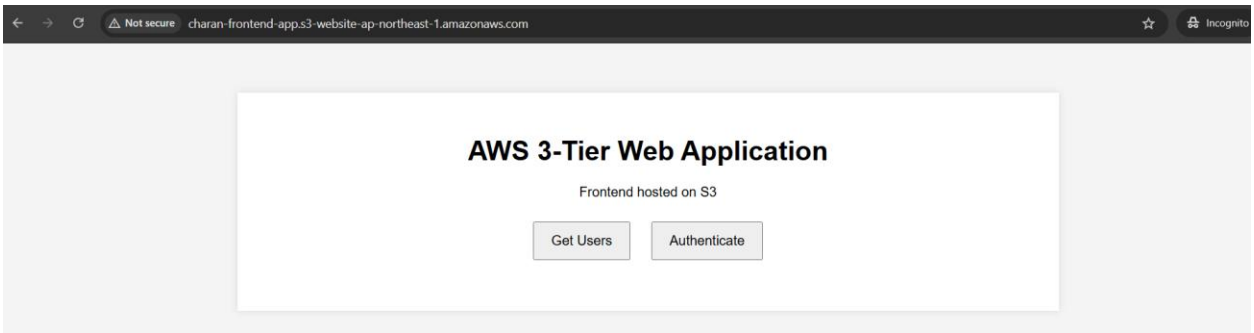


STEP 11 — VALIDATION

Test Frontend

Open:

S3 website endpoint

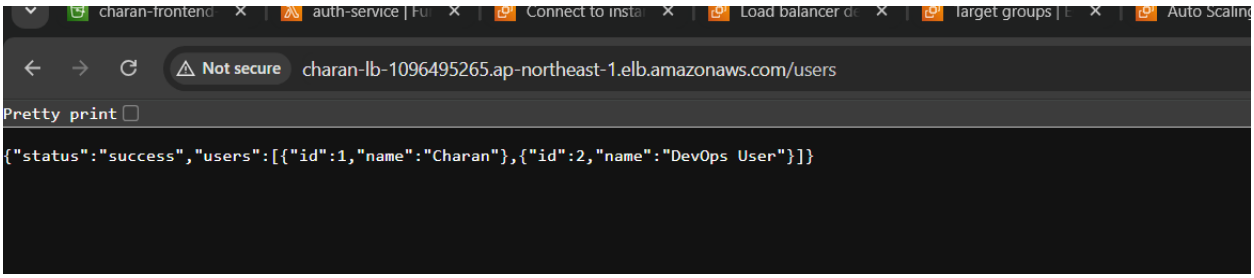


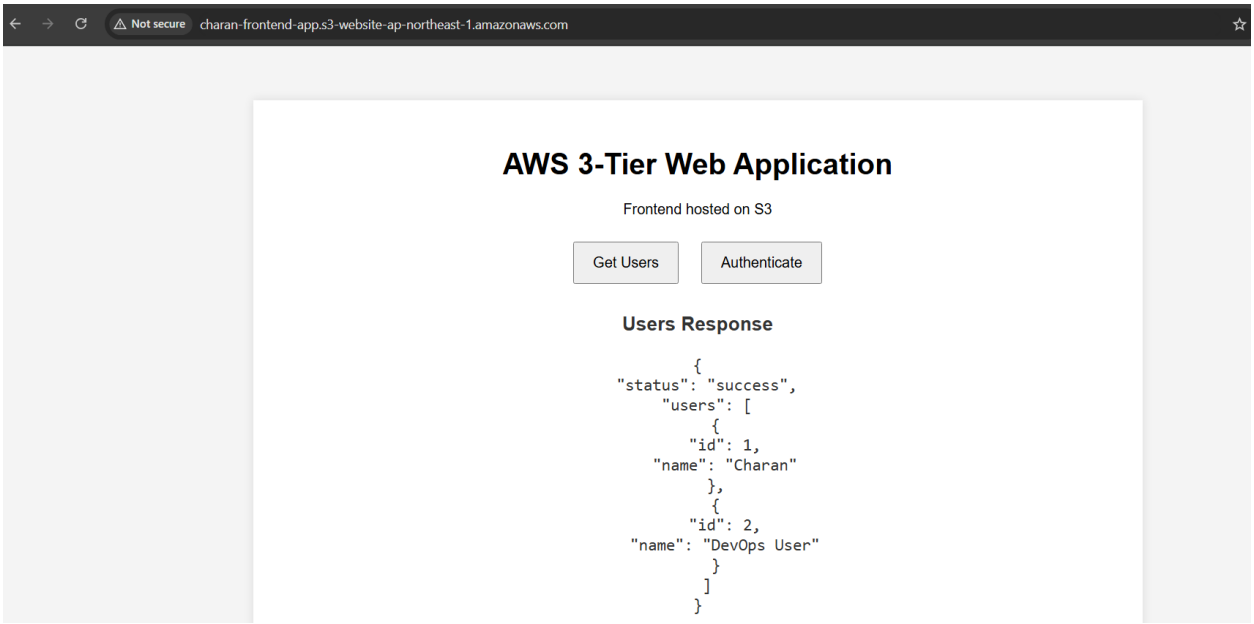
Test ALB Routing

http://ALB-DNS/users
http://ALB-DNS/auth
Expected:

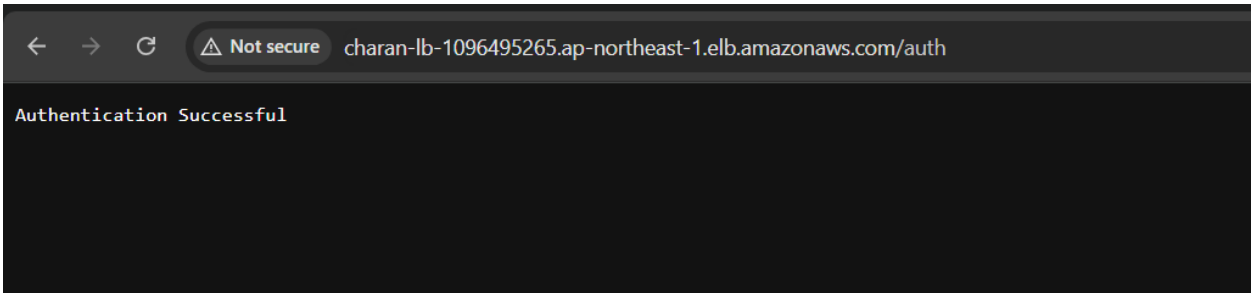
/users → Users API Response
/auth → Authentication Successful

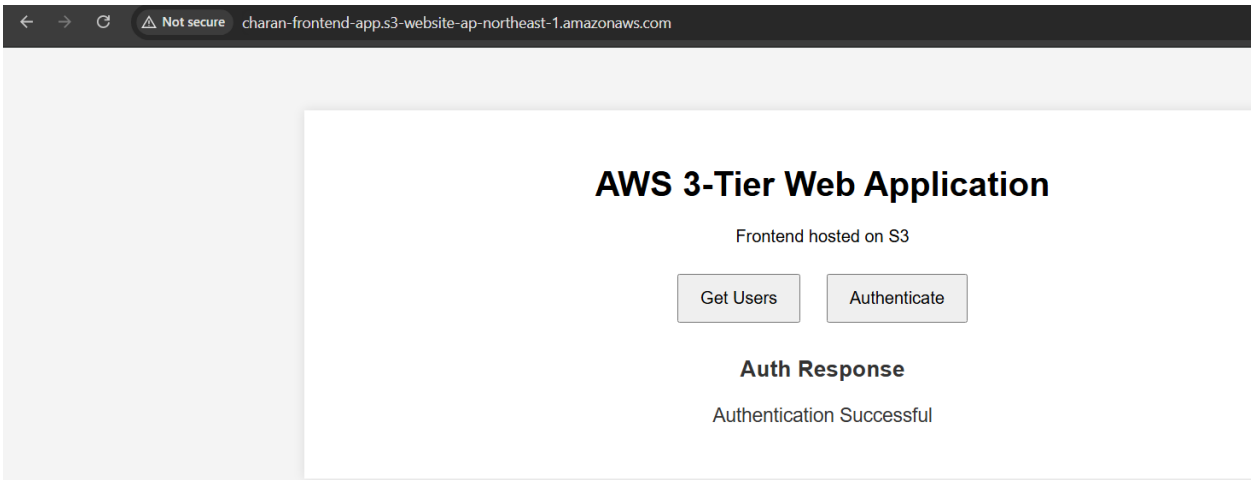
For Users response:





For Authentication:





Test Auto Scaling

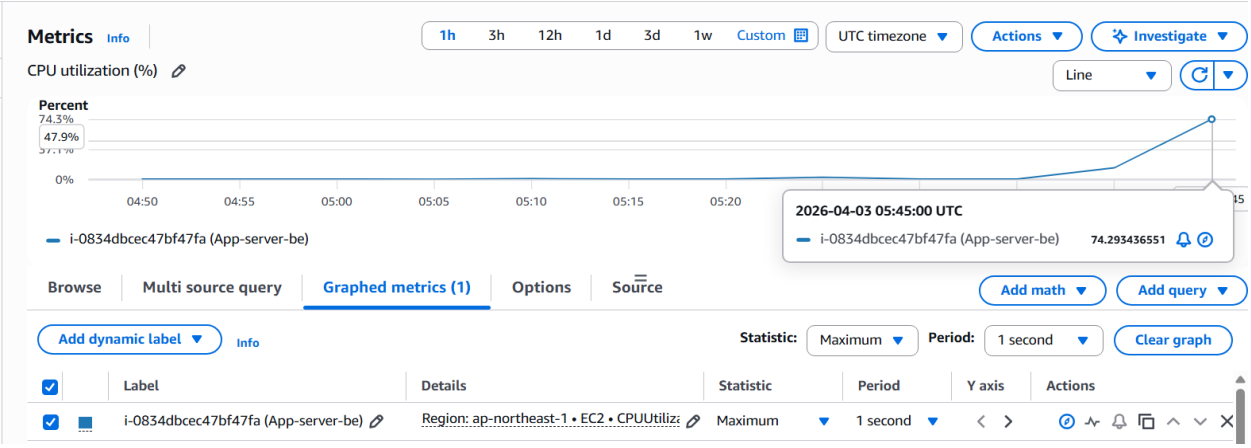
Run CPU stress:

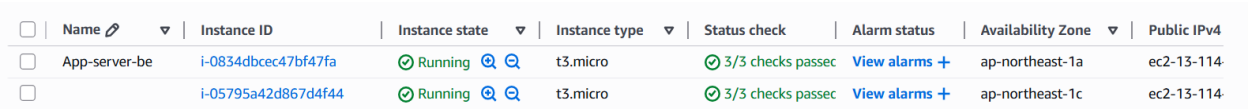
```
sudo yum install -y stress

stress --cpu 2 --timeout 300
```

```
[ec2-user@ip-172-31-41-176 ~]$ sudo yum install -y stress
Last metadata expiration check: 19:34:09 ago on Thu Apr  2 10:08:31 2026.
Dependencies resolved.
================================================================================
 Package          Architecture      Version                        Repository
================================================================================
Installing:
 stress           x86_64            1.0.7-2.amzn2023.0.1           amazonlinux
```

```
[ec2-user@ip-172-31-41-176 ~]$ stress --cpu 4 --timeout 3000
stress: info: [106760] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
```

Check scaling activity.





Final Validation Checklist

✓ S3 frontend working
✓ CORS working
✓ ALB routing working
✓ Lambda integration working
✓ EC2 backend working
✓ Auto scaling working
✓ Spread placement configured
✓ Secondary ENI attached
✓ NLB theory documented
✓ GWLB theory documented